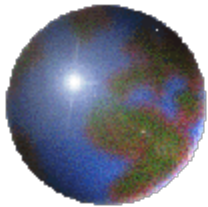


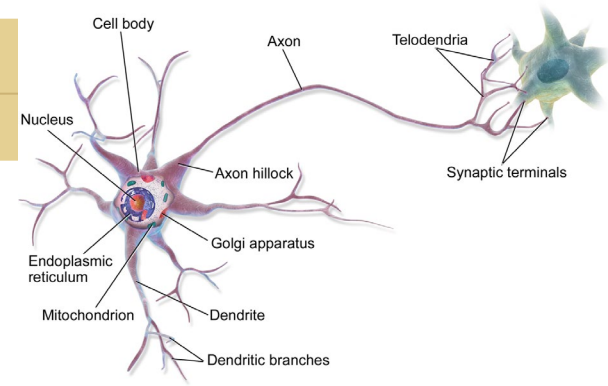
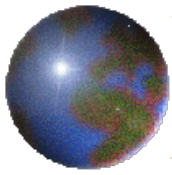
Supervised Learning: Artificial Neural Networks (ANN)



CWRU Quants Club Meeting

*Weatherhead School of Management
Case Western Reserve University*

*Joon Woo Bae
March 2026*



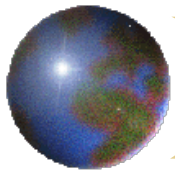
ANN (Artificial Neural Network)

History:

- Introduced in 1943 by McCulloch & Pitts with a simplified model of biological neurons.
- Resurgence in recent years due to: Large data availability, Increased computing power (thanks to GPUs and cloud computing), Improved training algorithms, etc.

Biological Neurons

- Composed of:
 - Cell body, dendrites (input), and a long axon (output)
 - Synapses connect neurons via neurotransmitters
- Behavior: When stimulated enough, a neuron fires an electrical impulse.
- Neurons are interconnected in large networks, enabling complex computations from simple units



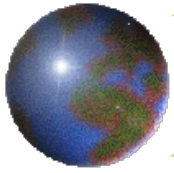
ANN (Artificial Neural Network),

✿ Value of House Example

- ✿ A linear regression model assumes that the relationship between each feature and price is straight-line (e.g., every extra 100 sq ft adds exactly \$10,000 to the price, regardless of neighborhood or other factors).
- ✿ But in reality, the effect of size might interact with location (e.g., 100 sq ft in New York City is worth much more than in a rural area), or have diminishing returns (e.g., going from 500 to 600 sq ft matters more than from 2,000 to 2,100).

✿ A neural network can automatically learn these:

- ✿ Nonlinear effects (e.g., price increases more rapidly in some ranges than others)
- ✿ Interactions between variables (e.g., size matters more in some neighborhoods)
- ✿ Thresholds and plateaus (e.g., more rooms help up to 4 rooms, then no added benefit)



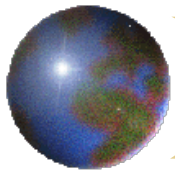
ANN (Artificial Neural Network) cont.

⊕ Advantages

- ⊠ Captures complex nonlinear relationships effectively
- ⊠ Performs well with large datasets and many input features
- ⊠ Highly flexible, used in wide variety of areas in machine learning and AI, including reinforcement learning and NLP

⊕ Disadvantages

- ⊠ Poor interpretability – often considered a “black box”
- ⊠ Requires large datasets and significant tuning
- ⊠ Computationally expensive and slow



ANN (Artificial Neural Network) *cont.*

Input Layer

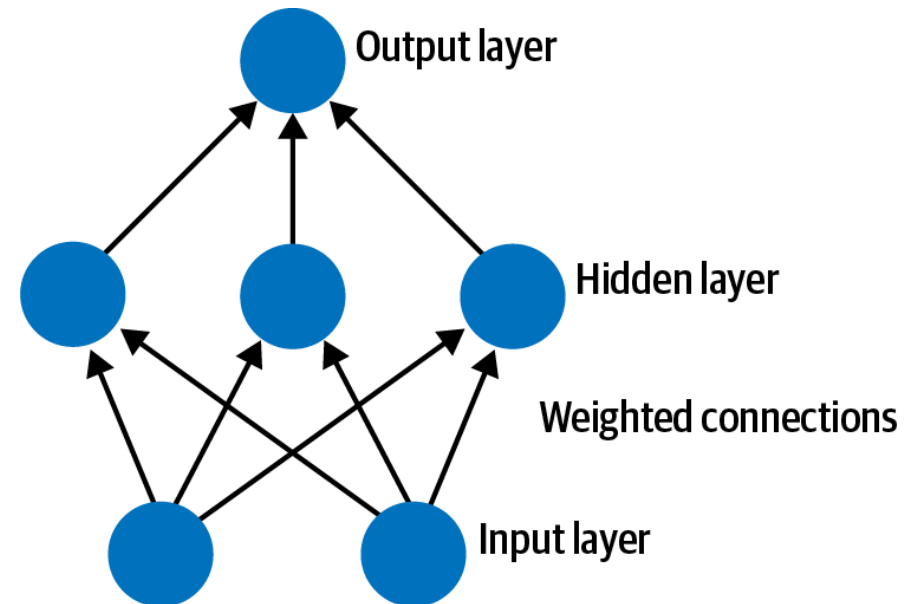
- ❑ Takes input directly from the dataset.
- ❑ Usually has one neuron per input feature.

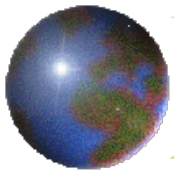
Hidden Layers

- ❑ Layers between input and output
- ❑ Add computational power and flexibility to the model.
- ❑ A network with many hidden layers (typically >3) is called a Deep Neural Network (DNN).
- ❑ More layers = more mathematical operations = higher computational cost.

Output Layer

- ❑ Produces the final output of the network.
- ❑ Output format depends on the specific task (e.g., classification or regression).



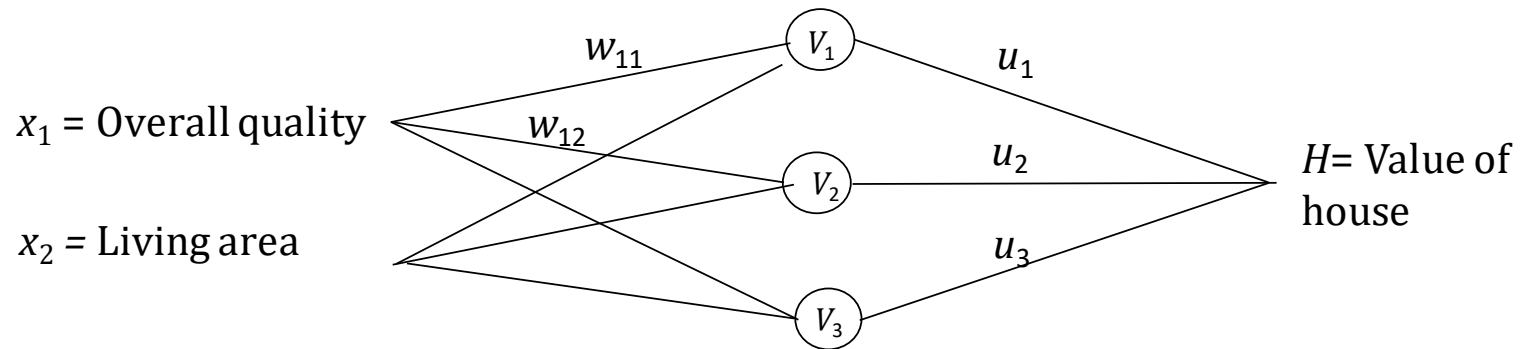


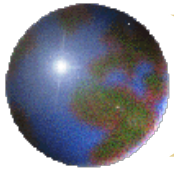
A Simple ANN (Artificial Neural Network)

Features

Neurons

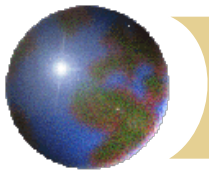
Target





Activation Functions

- ⊕ H is not related directly to the features
- ⊕ There is a hidden layer with neurons
- ⊕ H is related to the V_k
- ⊕ The V_k are related to the features
- ⊕ Activation functions define the relationships



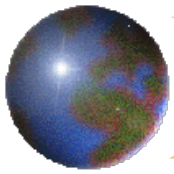
One possible set of equations for the house price example (13 parameters)

$$v_1 = \frac{1}{1 + \exp(-a_1 - w_{11}x_1 - w_{21}x_2)}$$

$$v_2 = \frac{1}{1 + \exp(-a_2 - w_{12}x_1 - w_{22}x_2)}$$

$$v_3 = \frac{1}{1 + \exp(-a_3 - w_{13}x_1 - w_{23}x_2)}$$

$$H = c + u_1v_1 + u_2v_2 + u_3v_3$$



Another Example, 19 parameters

Features

Neurons

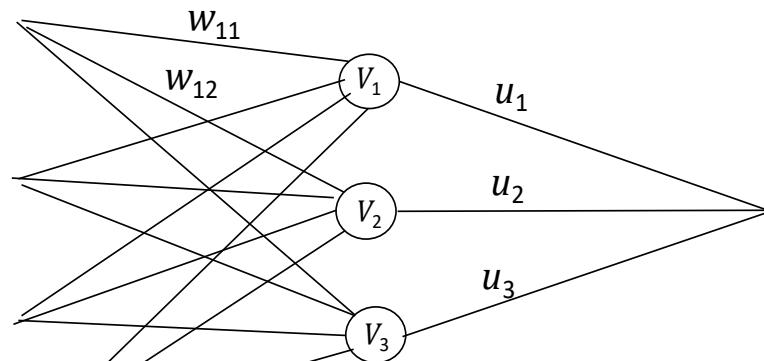
Target

x_1 = credit score

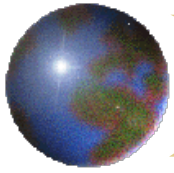
x_2 = income

x_3 = debt to income

x_4 = 1 if owns; 2 if rents



Q = Probability
of good loan



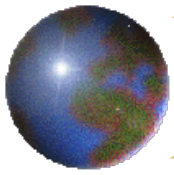
One possible set of equations for the loan example

$$v_1 = \frac{1}{1 + \exp(-a_1 - w_{11}x_1 - w_{21}x_2 - w_{31}x_3 - w_{41}x_4)}$$

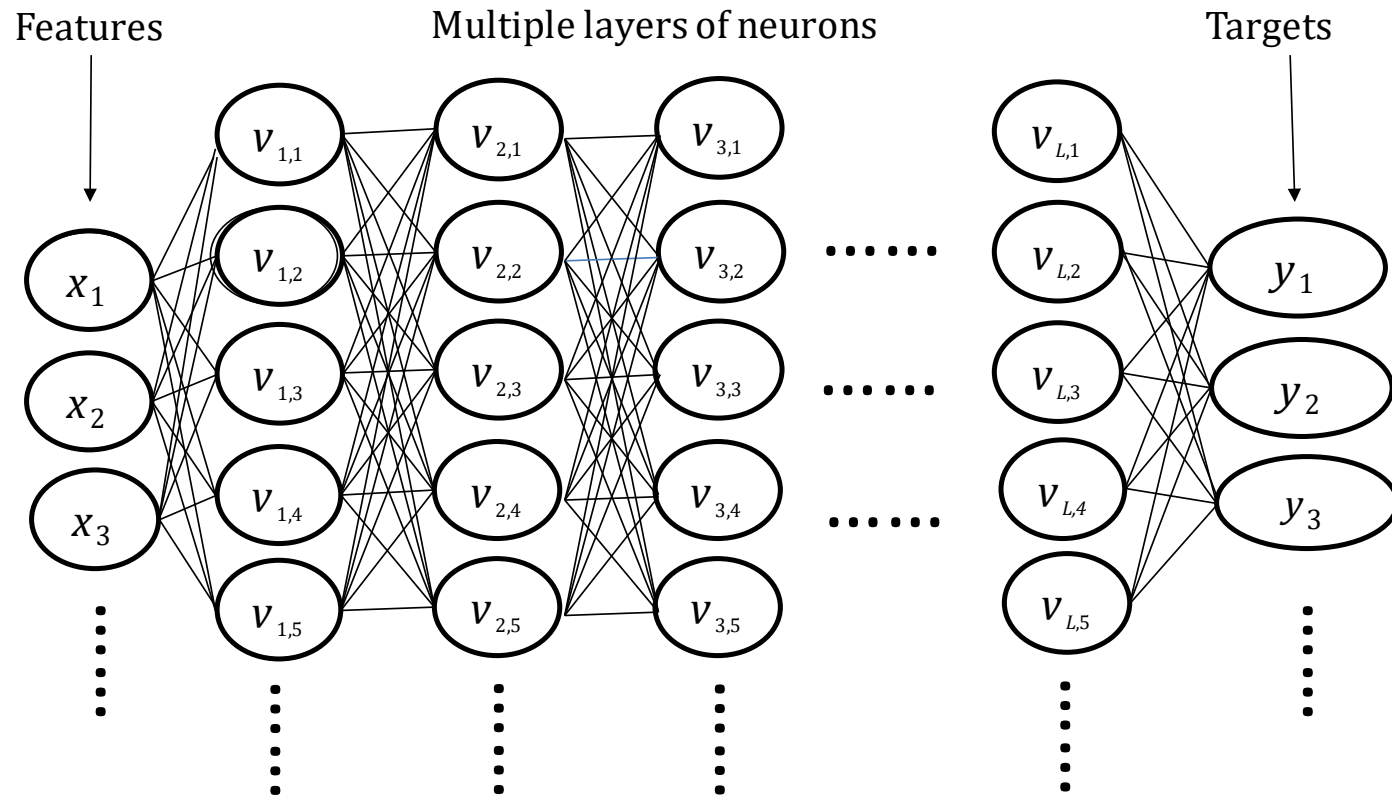
$$v_2 = \frac{1}{1 + \exp(-a_2 - w_{12}x_1 - w_{22}x_2 - w_{32}x_3 - w_{42}x_4)}$$

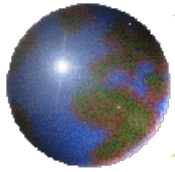
$$v_3 = \frac{1}{1 + \exp(-a_3 - w_{13}x_1 - w_{23}x_2 - w_{33}x_3 - w_{43}x_4)}$$

$$Q = \frac{1}{1 + \exp(-c - u_1V_1 - u_2V_2 - u_3V_3)}$$



A General Neural Network





Activation Functions

- ✚ An activation function relates values at one neuron to a linear combination of values in the previous layer.

$$v_{k,j} = f(y) \text{ where } y = a + b_1 v_{k-1,1} + b_2 v_{k-1,2} + \dots$$

- ✚ Examples of activation functions are:

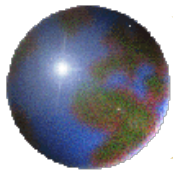
Linear: $f(y) = y$

Sigmoid: $f(y) = \frac{1}{1+e^{-y}}$ (gives values between 0 and 1)

Hyperbolic tangent: $f(y) = \frac{e^{2y}-1}{e^{2y}+1}$ (gives values between -1 and 1)

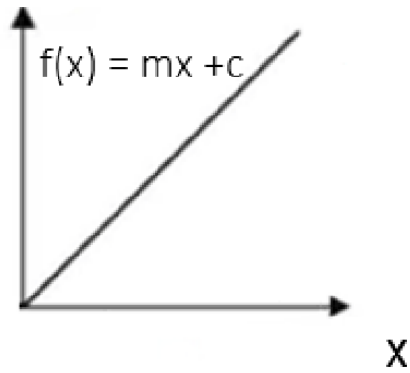
ReLU: $f(y) = \max(y, 0)$

Leaky ReLU: : $f(y) = \max(y, 0)$ if $y \geq 0$ and $\max(\alpha y, 0)$ if $y < 0$

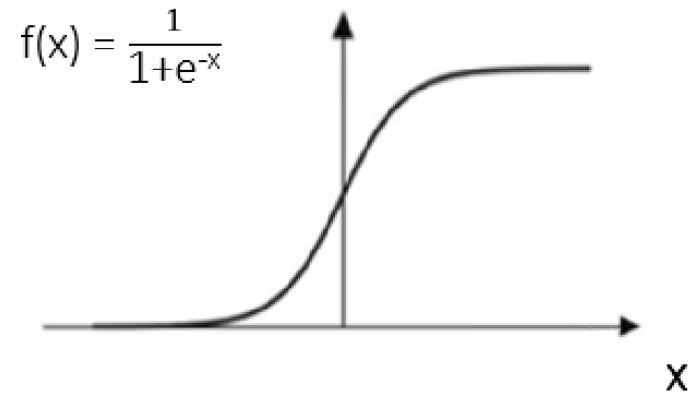


Activation Functions continued

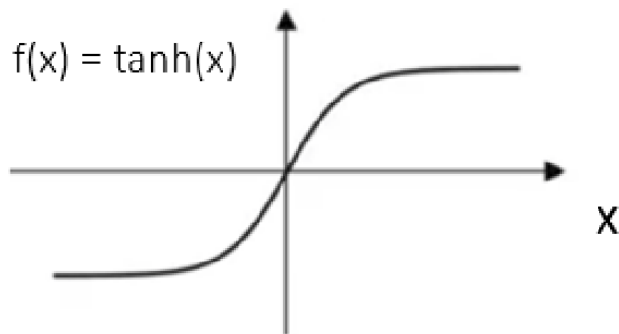
Linear activation function



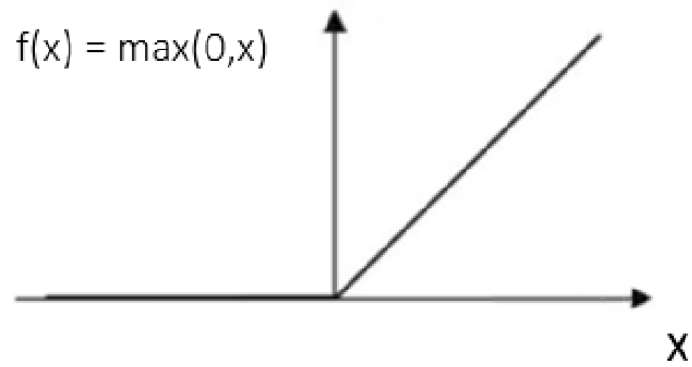
Sigmoid activation function

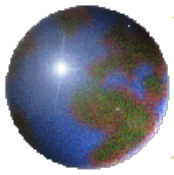


Tanh activation function



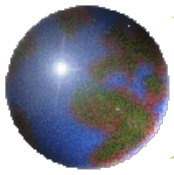
ReLU activation function





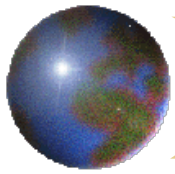
Activation Functions continued

- ✚ The same activation function is typically used throughout the network except possibly when relating the values in the output layer to values in the previous layer.
- ✚ When the target is a numerical estimate the final activation function is usually $f(y) = y$.
- ✚ In classification, the output layer is typically the probability of a positive outcome. The sigmoid function is then usually used to relate the values in the output layer to values in the previous layer
- ✚ Objective is to choose weights and biases to minimize mse or mae in the case of a prediction or to maximize likelihood in the case of a classification



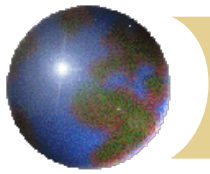
Universal Approximation Theorem

- ✚ Any continuous function can be approximated to arbitrary accuracy with one hidden layer (See K. Hornik: *Neural Networks*, 1991, 4:251-257)
- ✚ But this may require a very large number of neurons.
- ✚ Using several hidden layers can be computationally more efficient



Number of Parameters

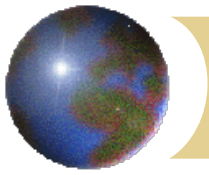
- ⊕ Neural networks can have a very large number of parameters
- ⊕ If there are F features, H hidden layers, M neurons in each hidden layer and T targets the number of parameters is
$$(F+1)M + M(M+1)(H-1) + (M+1)T$$
- ⊕ The simple house price neural network has 13 parameters and the simple loan decision neural network has 19 parameters
- ⊕ But in practice a small neural network might have 4 features, 3 hidden layers, 30 neurons per layer and one target for a total of 2,041 parameters.
- ⊕ Larger neural networks have tens, or even hundreds, of thousands of parameters



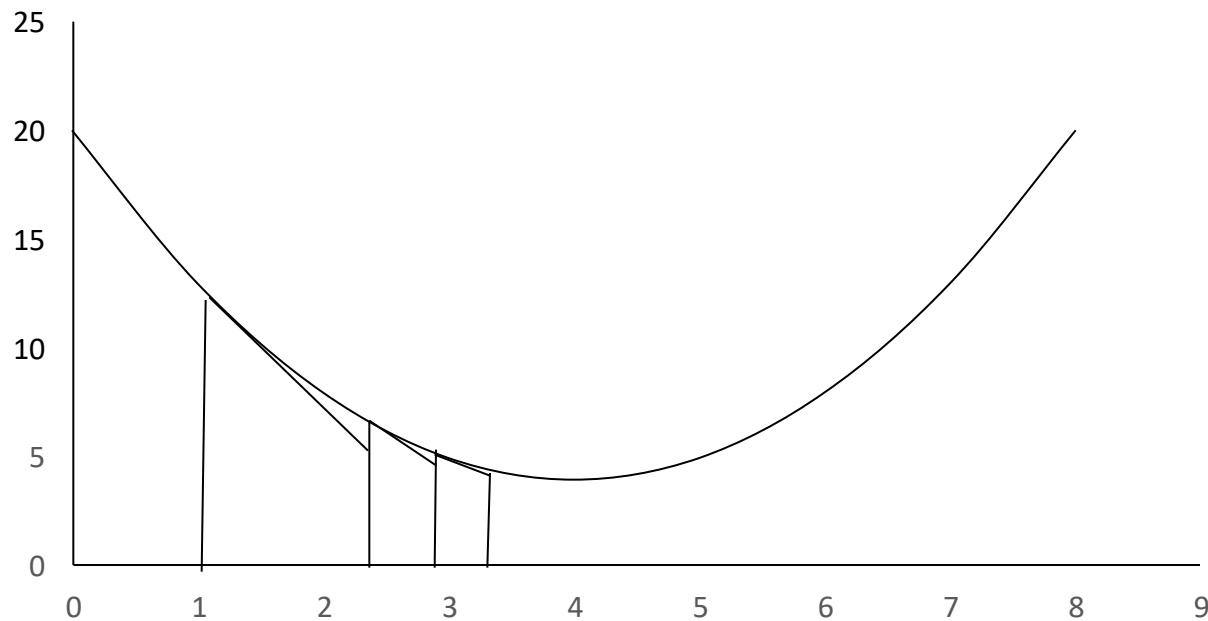
Gradient Descent and Neural Nets

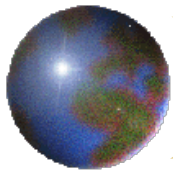
- ✚ Suppose that there are N parameters. An objective function (e.g., mse or one based on maximum likelihood estimates) is minimized for training set data
- ✚ A gradient descent algorithm starts with a set of values for the N parameters, calculates the direction of steepest descent down the valley, takes a step, calculates a new direction of steepest descent, takes another step, and so on
- ✚ The partial derivatives with respect to the parameters are calculated by a procedure known as backpropagation. This involves working back through the network using the chain rule. (See Rumelhart, Hinton, and Williams, *Nature*, 1986, 323, 533-536)
- ✚ The size of the step is determined by the “learning rate”. If the step is too small the algorithm will be very slow. If it is too large there are liable to be oscillations:

$$\text{Increase in variable} = -\text{learning rate} \times \text{gradient}$$



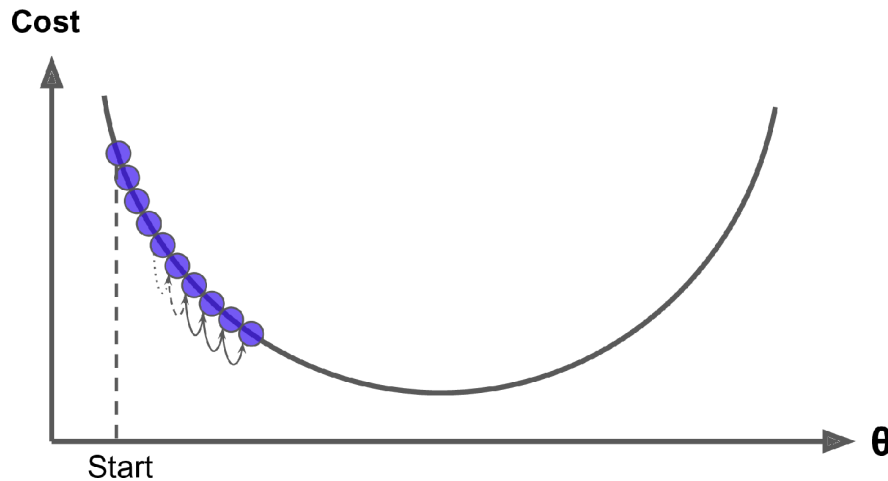
Simple Example: Calculating the value of x that minimizes y when $y=x^2-8x+20$



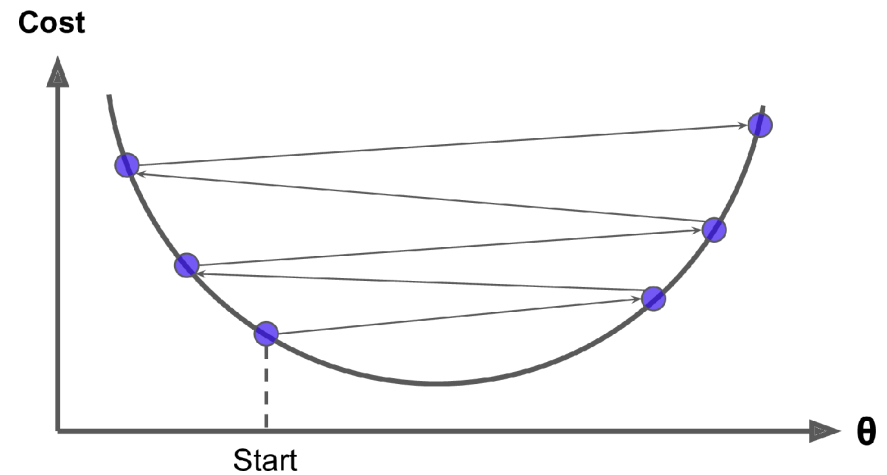


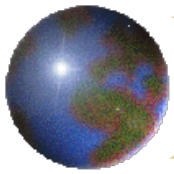
Gradient Descent and Learning Rate

✚ If Learning Rate is too small



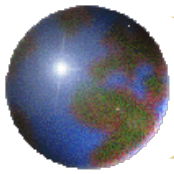
✚ If Learning Rate is too large





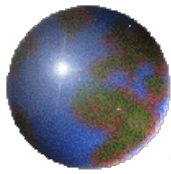
When Learning Rate is 0.2

Iteration	x	Gradient	Change in x
0	1.000	-6.000	1.200
1	2.200	-3.600	0.720
2	2.920	-2.160	0.432
3	3.352	-1.296	0.259
4	3.611	-0.778	0.156
5	3.767	-0.467	0.093
6	3.860	-0.280	0.056
7	3.916	-0.168	0.034
8	3.950	-0.101	0.020
9	3.970	-0.060	0.012
10	3.982	-0.036	0.007
11	3.989	-0.022	0.004
12	3.993	-0.013	0.003
13	3.996	-0.008	0.002



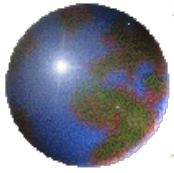
When Learning Rate is 0.02

Iteration	x	Gradient	Change in x
0	1.000	-6.000	0.120
1	1.120	-5.760	0.115
2	1.235	-5.530	0.111
3	1.346	-5.308	0.106
4	1.452	-5.096	0.102
5	1.554	-4.892	0.098
6	1.652	-4.697	0.094
7	1.746	-4.509	0.090
8	1.836	-4.328	0.087
9	1.922	-4.155	0.083
10	2.006	-3.989	0.080
11	2.085	-3.829	0.077
12	2.162	-3.676	0.074
13	2.235	-3.529	0.071



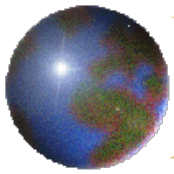
When Learning Rate is 1.2

Iteration	x	Gradient	Change in x
0	1.000	-6.000	7.200
1	8.200	8.400	-10.080
2	-1.880	-11.760	14.112
3	12.232	16.464	-19.757
4	-7.525	-23.050	27.660
5	20.135	32.269	-38.723
6	-18.589	-45.177	54.213
7	35.624	63.248	-75.898
8	-40.274	-88.547	106.257
9	65.983	123.966	-148.760
10	-82.776	-173.553	208.263
11	125.487	242.974	-291.569
12	-166.082	-340.163	408.196
13	242.114	476.229	-571.475



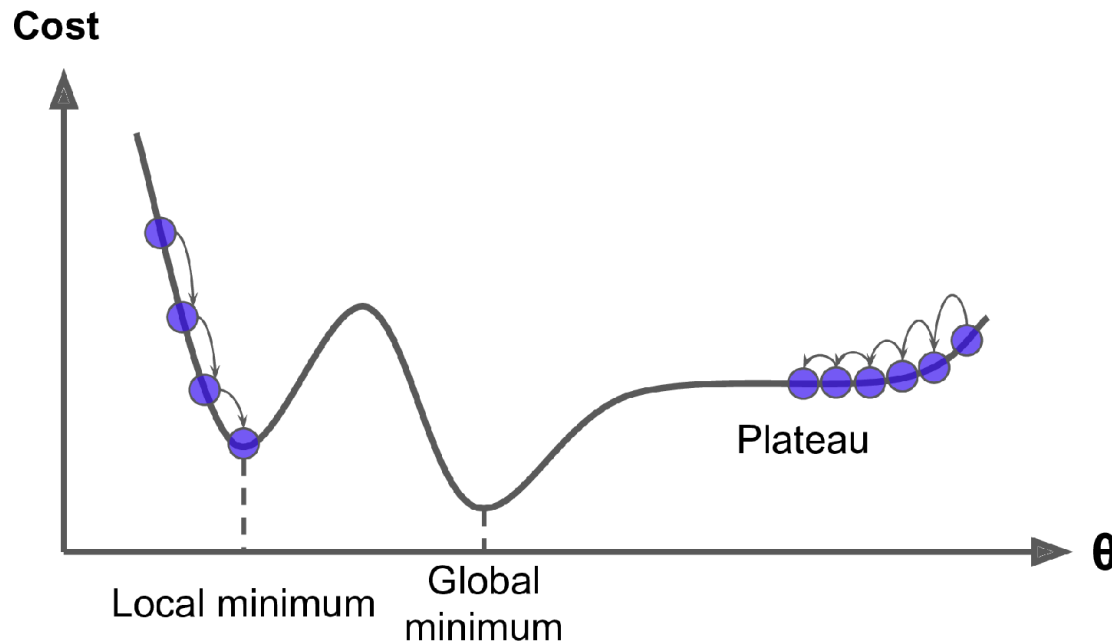
Stopping Rule

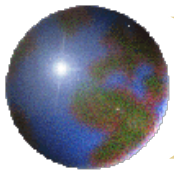
- ⊕ Because applications have many parameters it is important to use a stopping rule to avoid over-fitting
- ⊕ We calculate results for the validation set at the same time as the training set.
- ⊕ When the results for the validation set start to get worse we stop



Gradient Descent. *cont.*

- ✚ Be careful with...
 - ✚ Irregular Cost Surfaces: Real-world cost functions often have complex shapes—holes, ridges, and flat plateaus—not simple, bowl-like curves.
 - ✚ Slow Convergence: Flat regions (plateaus) can cause very slow progress, and stopping early may lead to suboptimal solutions
 - ✚ **Local vs Global Minimum:** The algorithm may get stuck in a local minimum instead of reaching the global minimum.

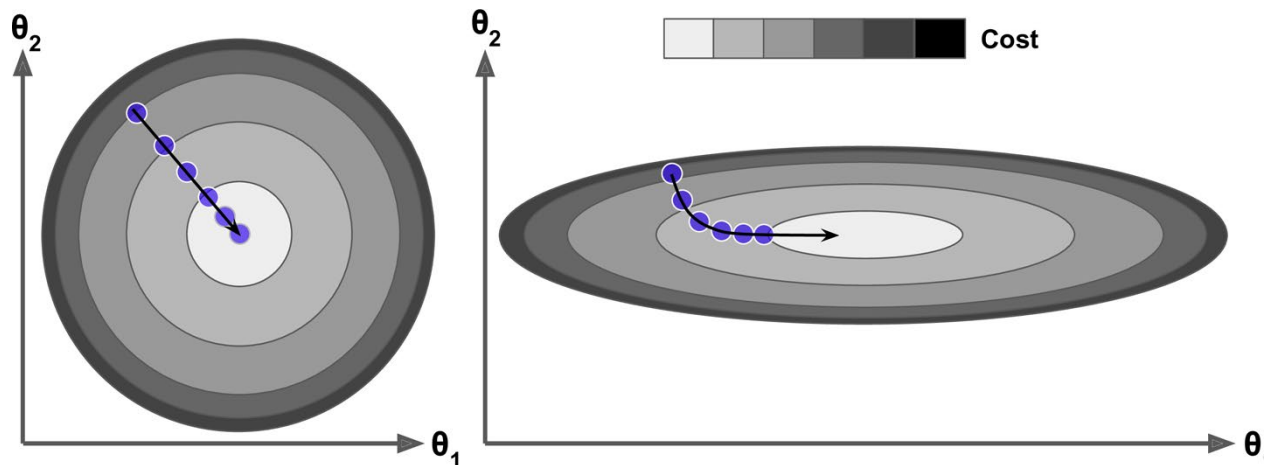


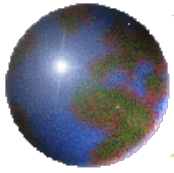


Gradient Descent. *cont.*

✚ Be careful with...

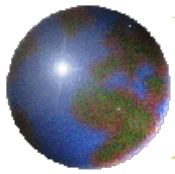
- ✚ Irregular Bowl Shape of Cost Function: Gradient Descent works best when the cost function resembles a symmetrical bowl.
- ✚ Effect of Unequal Feature Scales: If features have very different scales, the cost surface becomes elongated, making descent inefficient.





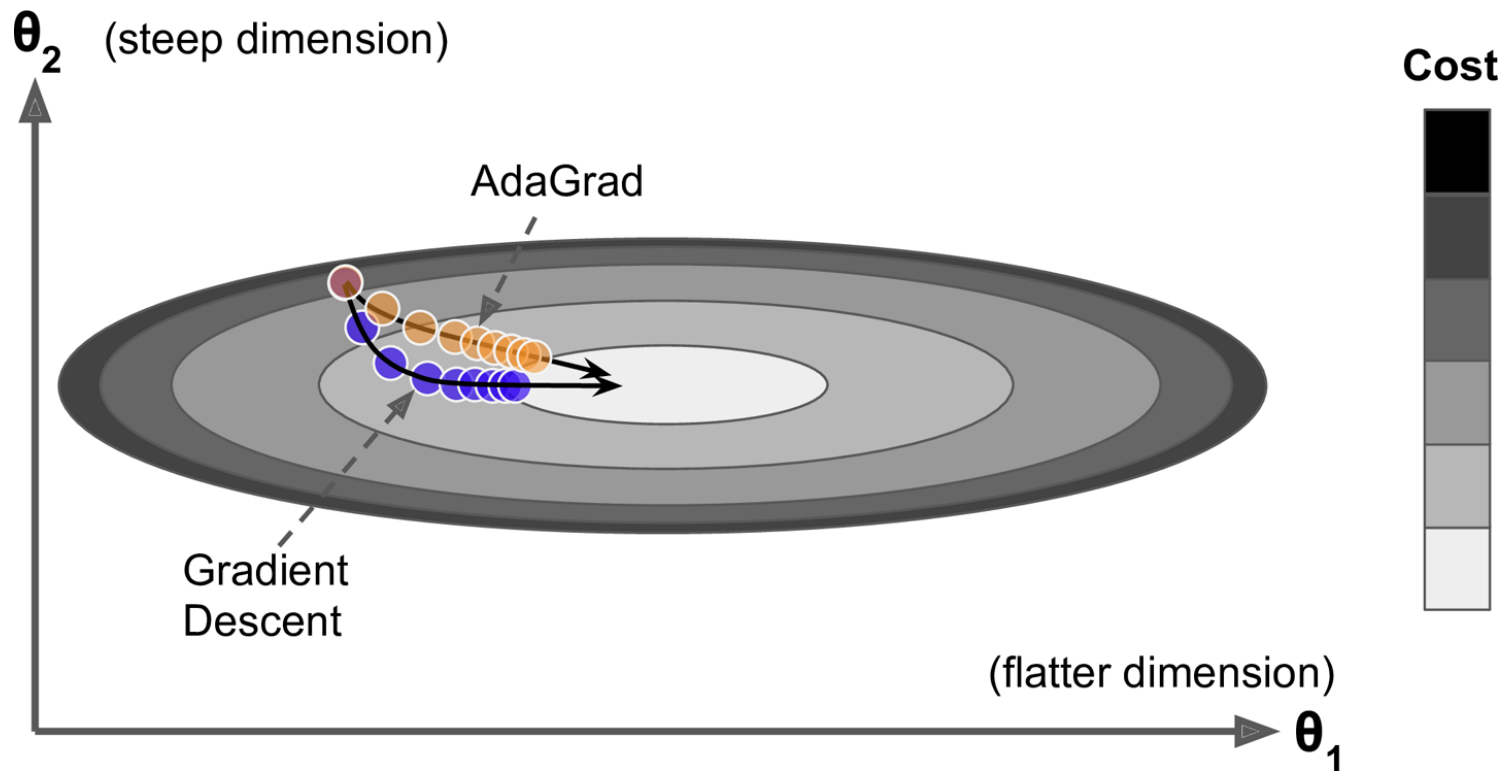
Other Details

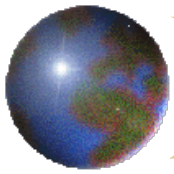
- ✚ Use mini batches of data to calculate gradients. (One epoch is one complete use of training set.)
- ✚ Use a momentum strategy
- ✚ Adaptively calculate the learning rate (Adam)
- ✚ Learning rate decay
- ✚ Dropouts



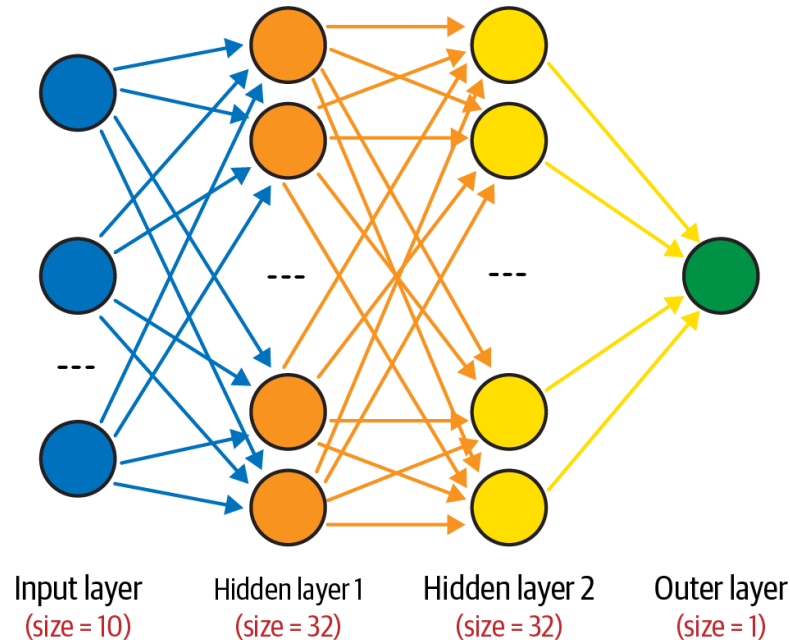
Adaptive Learning Rate (example)

- ✚ In short, this algorithm decays the learning rate, but it does so faster for steep dimensions than for dimensions with gentler slopes. It helps point the resulting updates more directly toward the global optimum.

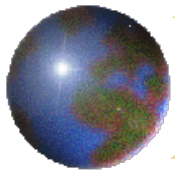




ANN example: Python Implementation

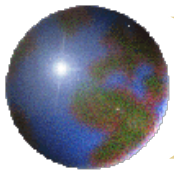


```
from Tensorflow.Keras.models import Sequential
from Tensorflow.Keras.layers import Dense
model = Sequential()
model.add(Dense(32, input_dim=10, activation='relu' ))
model.add(Dense(32, activation='relu' ))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=[ 'accuracy' ])
model.fit(data, Y, nb_epoch=10, batch_size=32)
```

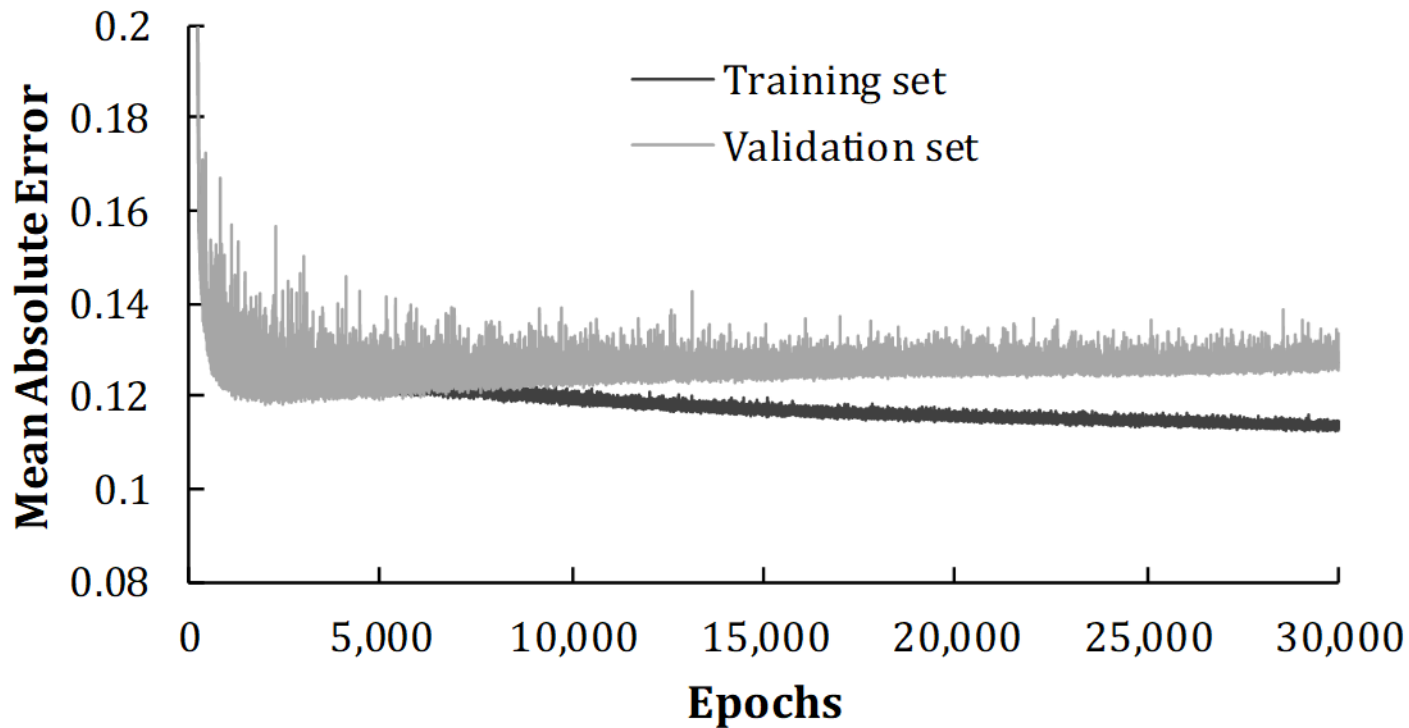


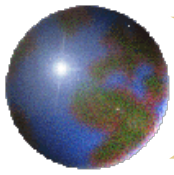
Black-Scholes-Merton Application

- ✚ We generated 10,000 call option prices using the Black-Scholes-Merton model and then added a normally distributed error of 0.15 to the price.
- ✚ The parameters were sampled randomly from uniform distributions
- ✚ The model had three hidden layers and 20 neurons per layer

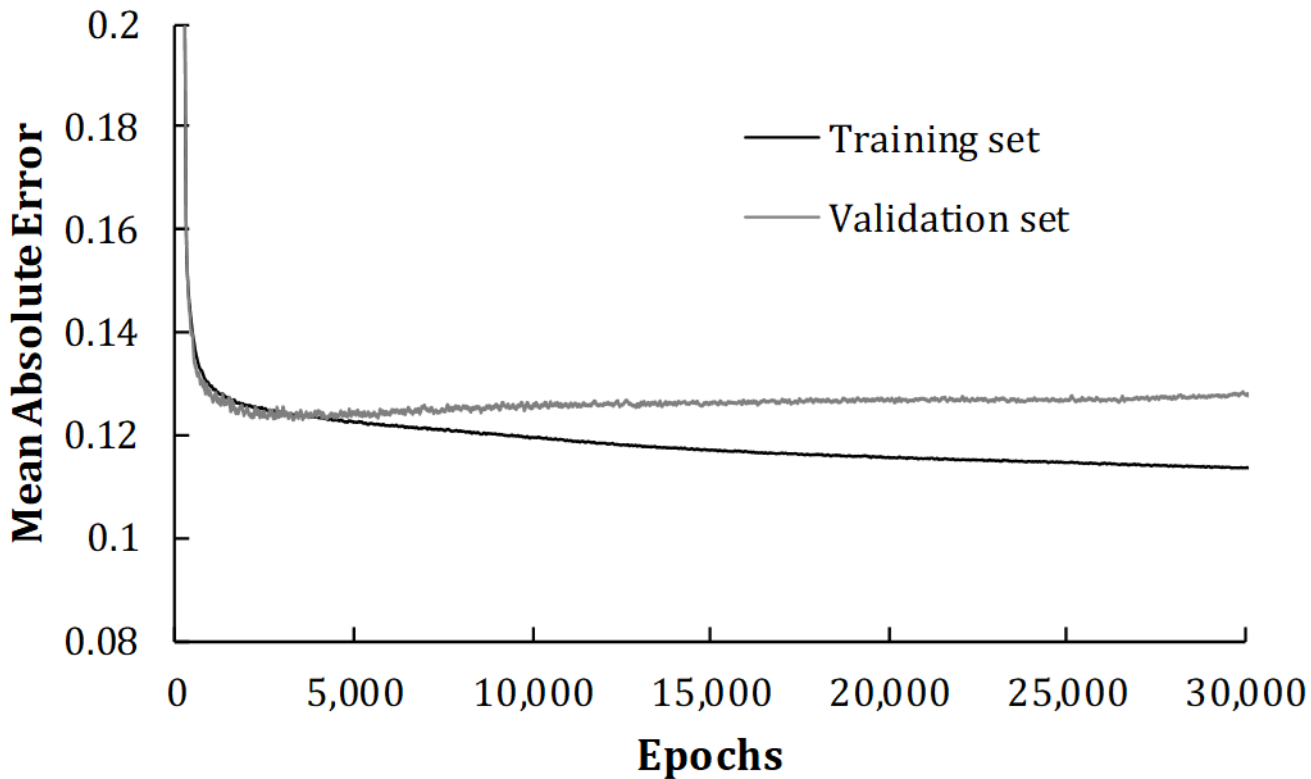


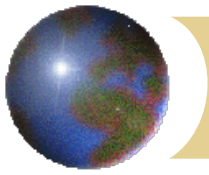
Training set and validation set mse as the epochs of training is increased





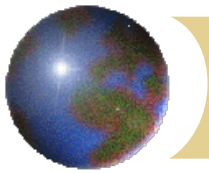
Smoothed MSE (Moving Average over 50 epochs) Stop after 2575 epochs





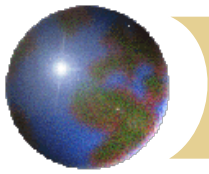
Results

- ⊕ With only 10,000 observations the neural network imitated the Black-Scholes-Merton model well
- ⊕ It eliminated much of the random noise we added to the BSM prices.



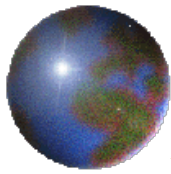
Using a similar idea to value exotic derivatives

- ⊕ Some “exotic” derivatives are valued using Monte Carlo simulation which is slow
- ⊕ Neural networks can be used as follows:
 - ⊠ Do an initial analysis to generate a large amount of data relating prices to input variables
 - ⊠ Construct a neural network to replicate prices
 - ⊠ Obtain fast pricing by working forward through network
- ⊕ Useful for scenario analysis



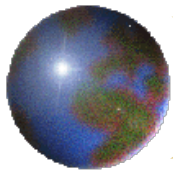
Another Application of ANN: Understanding implied volatilities

- ✚ The implied volatility of an option is the volatility that, when substituted into Black-Scholes-Merton formula, gives the price of the option in the market

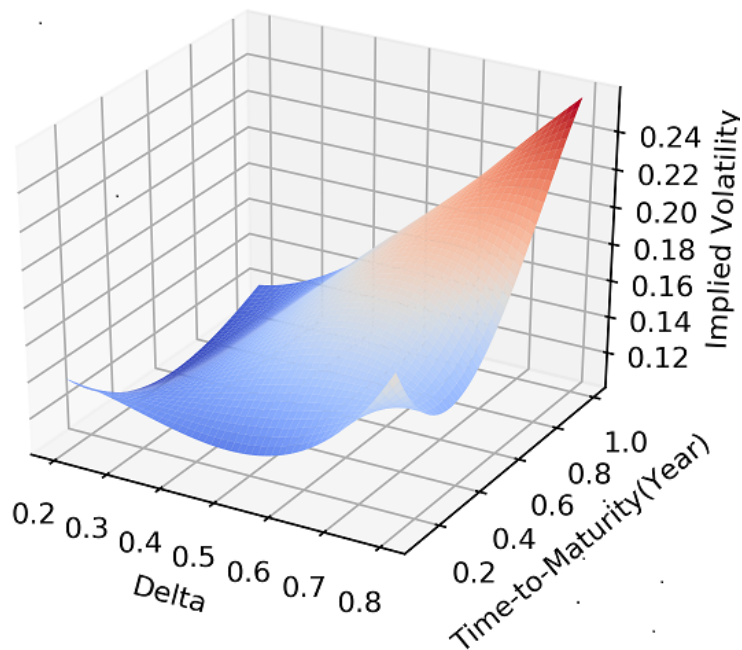


Implied volatilities continued

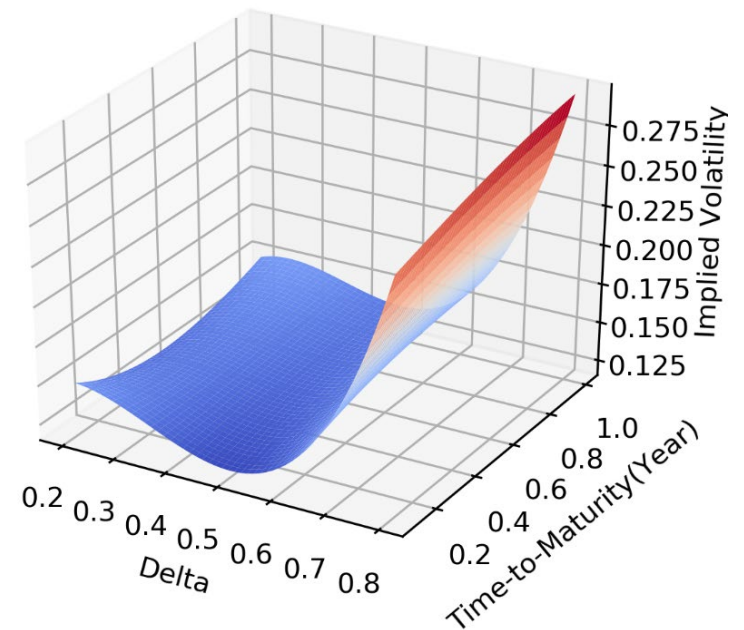
- ⊕ If Black-Scholes-Merton was used for pricing, all options on an asset would have the same implied volatility
- ⊕ In fact, there is quite a variation in implied volatilities
- ⊕ Nevertheless implied volatilities are used to communicate prices and it is therefore important for traders to monitor implied volatilities
- ⊕ The volatility surface shows implied volatilities as a function of
 - ⊠ Moneyness or how likely the option is to be exercised (Traders measure this by the delta, the sensitivity of the option to the underlying asset's price)
 - ⊠ Time to maturity, T



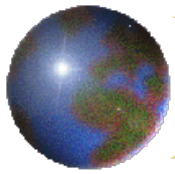
Volatility Surfaces for S&P 500



Jan 31, 2019

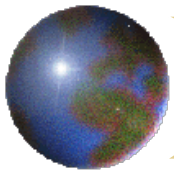


June 25, 2019



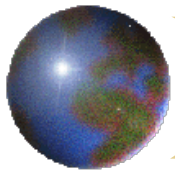
Understanding Volatility Surface Movements

- ⊕ To understand volatility surface movements we used data on S&P 500 call options to construct a neural network
- ⊕ Input layer:
 - ⊠ Daily asset price return
 - ⊠ Moneyness (measured by delta)
 - ⊠ Time to maturity
- ⊕ Output layer:
 - ⊠ Change in implied volatility



Details

- ⊕ 3 hidden layers
- ⊕ 20 neurons per layer
- ⊕ Observations from 2014-2019
- ⊕ Randomly sampled 100 options per day
- ⊕ 125,700 options in total
- ⊕ 60% for training set
- ⊕ 20% for validation set
- ⊕ 20% for test set
- ⊕ Z-score scaling

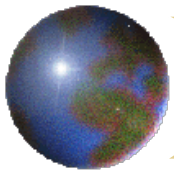


Sample of raw data

<i>Date</i>	<i>Return (%) on S&P 500</i>	<i>Maturity (years)</i>	<i>Delta</i>	<i>Change in implied volatility (bps)</i>
Jan 28, 2015	0.95	0.608	0.198	-31.1
Sept 8, 2017	1.08	0.080	0.752	-54.9
Jan 24, 2018	0.06	0.956	0.580	-1.6
Jun 24, 2019	-0.95	2.896	0.828	40.1

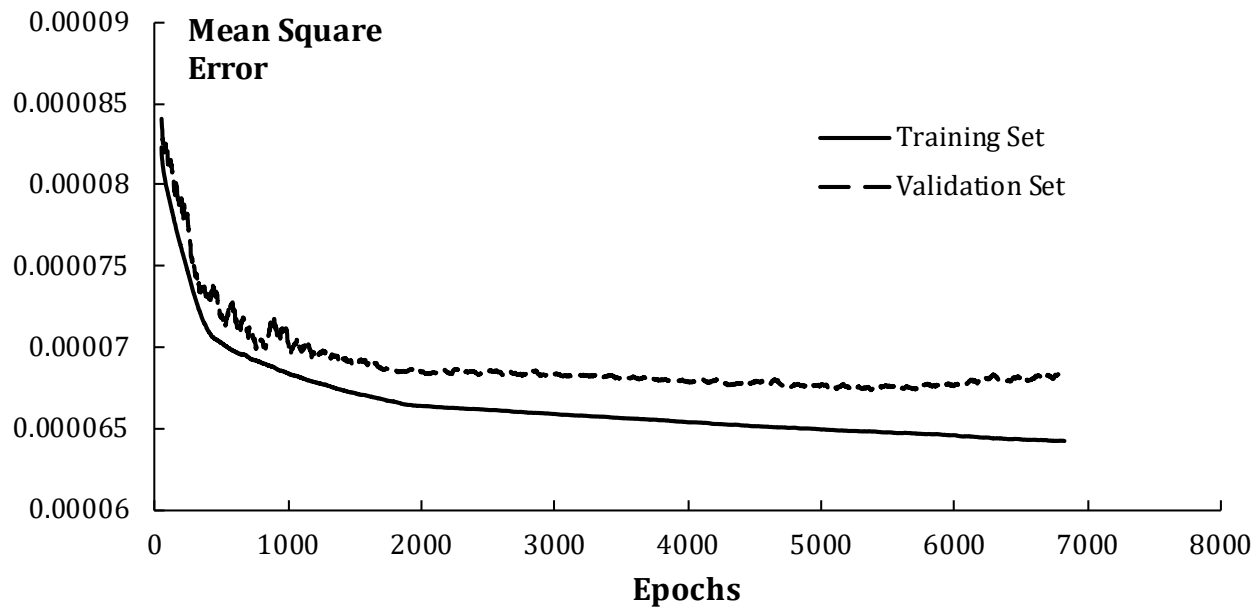
After scaling:

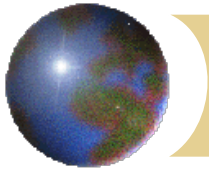
<i>Date</i>	<i>Return (%) on S&P 500</i>	<i>Maturity (years)</i>	<i>Delta</i>	<i>Change in implied volatility (bps)</i>
Jan 28, 2015	1.091	-0.102	-1.539	-31.1
Sept 8, 2017	1.247	-0.695	0.445	-54.9
Jan 24, 2018	0.027	0.289	-0.171	-1.6
Jun 24, 2019	-1.176	2.468	0.716	40.1



Mean Squared Error

(Training stopped after 5,826 epochs)





Results

- ⊕ Test set gave a modest 14% improvement over a simple analytic model proposed by Hull and White in 2017
- ⊕ However, when the VIX index on Day t was used as a feature to predict changes between Day t and Day $t+1$ there was a considerable improvement
- ⊕ The behavior of the volatility surface is different in high and low volatility environments